

رسالة محمد

نام دانشگاه: مرکز فنی مهندسی بوئین زهرا

نام درس: طراحی زبان های برنامه نویسی

استاد: دکتر سمن شیشه چی

نام دانشجو: مرتضی رسولی فاضل

شماره دانشجویی: ۴۰۱۲۱۱۴۱۲۵

رشته تحصیلی: مهندسی کامپیوتر

مقطع تحصیل: کارشناسی

تاریخ تحویل: ۱۴۰۴/۵/۱۴

موضوع تحقیق: بررسی زبان پایتون از منظر طراحی زبان های برنامه نویسی

فهرست مطالب

مقدمه.....	۴
زبان پایتون یک زبان مفسری.....	۵
تفاوت زبان مفسری و کامپایلری.....	۵
پایتون زبان هم مفسری هم کامپایلری.....	۵
زمان انقیاد در پایتون.....	۶
انقیاد دیر رس در پایتون.....	۶
انقیاد زود رس در پایتون.....	۷
تبدیل نوع در پایتون.....	۸
تبدیل نوع ضمنی در پایتون.....	۸
تبدیل نوع صریح در پایتون.....	۸
چالش های پشتیبانی زبان از هر دو تبدیل نوع.....	۹
کنترل نوع در پایتون.....	۱۰
کنترل نوع پوبا در پایتون.....	۱۰
توصیف گر در پایتون.....	۱۱
نوع شمارش در پایتون.....	۱۲
انواع رشته های ورودی در پایتون.....	۱۳
کپسول سازی در پایتون.....	۱۴
مدیریت زباله در پایتون.....	۱۵
نگهداری رکورد ها در پایتون.....	۱۶
ساختار لیست در پایتون.....	۱۷

مقدمه

زبان برنامه‌نویسی پایتون (Python)، یکی از زبان‌های سطح بالا، تفسیری و شی‌گراست که در سال ۱۹۹۱ توسط خیدو فان روسوم (Guido van Rossum) طراحی و منتشر شد. هدف از توسعه این زبان، ارائه‌ی ابزاری ساده، خوانا و قدرتمند برای برنامه‌نویسان بوده که به راحتی بتوانند مسائل پیچیده را با کدهایی مختصر و قابل‌فهم حل کنند.

پایتون به دلیل سینتکس ساده و شباهت زیادش به زبان طبیعی، یکی از گزینه‌های محبوب در طراحی زبان‌های آموزشی و همچنین ابزار توسعه حرفه‌ای محسوب می‌شود. استفاده از مفاهیم مدرن طراحی زبان مانند تایپ پویا، مدیریت خودکار حافظه، و پشتیبانی از چند سبک برنامه‌نویسی (شی‌گرا، تابعی و دستوری) باعث شده پایتون به یک مرجع مهم در بررسی اصول طراحی زبان‌های برنامه‌نویسی تبدیل شود.

در این تحقیق تلاش شده است که ابعاد مختلف این زبان از منظر طراحی زبان بررسی کرد.

زبان پایتون یک زبان مفسری

تفاوت زبان مفسری و کامپایلری:

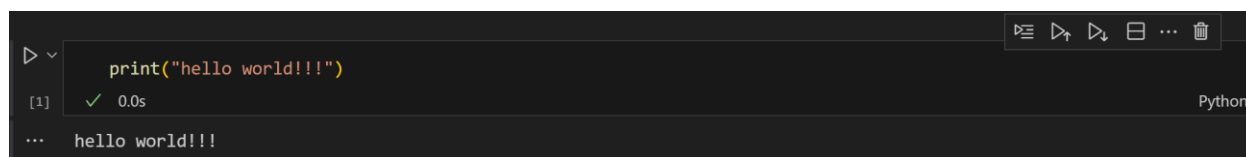
در زبان های مفسری کد های نوشته شده به صورت خط به خط خوانده و سپس اجرا میشوند. در این روش بخاطر اینکه خط به خط کد خوانده میشود سرعت کمتری نسبت به کامپایلری داریم ولی از سمت دیگر تشخیص خطا بخاطر وجود ویژگی خط به خط اجرا شدن برنامه راحت تر است.

از سمت دیگر زبان کامپایلری کد های داخل فایل را به صورت کامل اول به زبان ماشین تبدیلی میکند و سپس کد به صورت یکجا اجرا میشود. این ویژگی باعث میشود که سرعت برنامه بالا برود ولی از سمت دیگر خطایابی در این زبان ها سخت تر میشود و اگر قرار باشد که یک خط کد اصلاح شود، کل برنامه باید مجدد به زبان ماشین برگردد.

پایتون زبان هم مفسری هم کامپایلری:

پایتون به طور سنتی زبان مفسری شناخته می شود، اما در واقعیت، ترکیبی از هر دو است:

- ابتدا کد پایتون به بایت کد (فایل های .pyc) کامپایل می شود.
- سپس این بایت کد توسط ماشین مجازی پایتون (PVM) تفسیر و اجرا می شود.

A screenshot of a Python interpreter window. The window has a dark background with a toolbar at the top containing icons for running, stepping through code, and other debugging functions. Below the toolbar, the code `print("hello world!!!")` is entered. The output shows `[1] ✓ 0.0s` and `hello world!!!`. The word "Python" is visible in the bottom right corner of the window.

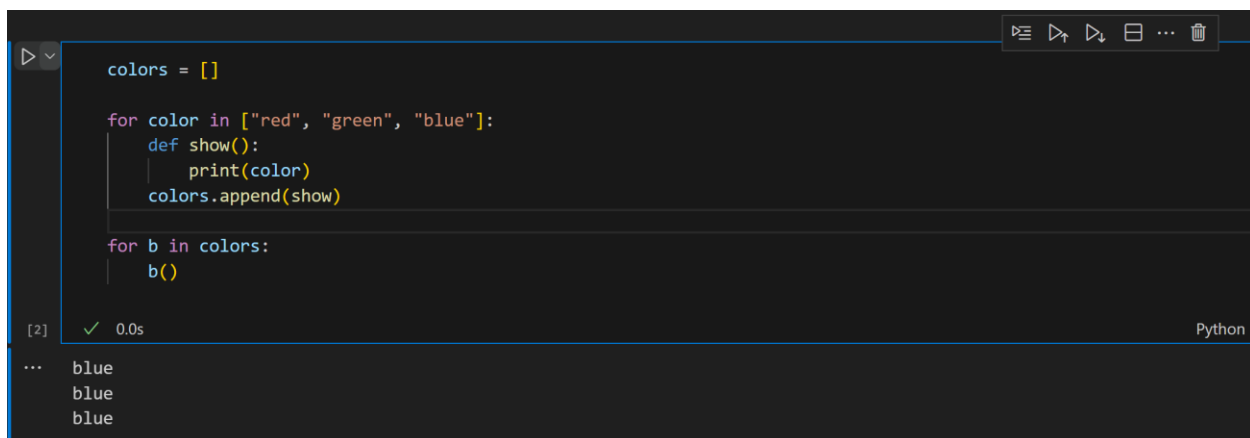
در قطعه کد بالا اول کد به شکل کامپایلری به بایت کد تبدیل میشود و سپس توسط PVM و به شکل تفسیری اجرا میشود.

زمان انقیاد در پایتون

زمان انقیاد به دو دسته زود رس و دیر رس تقسیم بندی میشود که در حالت زود رس یعنی در زمان کامپایل نوع و رفتار یک متغیر مشخص میشود و در حالت دیر رس به این شکل است که در زمان اجرا برنامه نوع متغیر مشخص میشود.

انقیاد دیر رس در پایتون

زبان پایتون به طور کلی از انقیاد دیر رس استفاده میکند. این به این معنی است که در زمان اجرا برنامه نوع متغیر مشخص میشود این رفتار زبان رو در کد زیر بهتر درک میکنیم:



```
colors = []

for color in ["red", "green", "blue"]:
    def show():
        print(color)
    colors.append(show)

for b in colors:
    b()
```

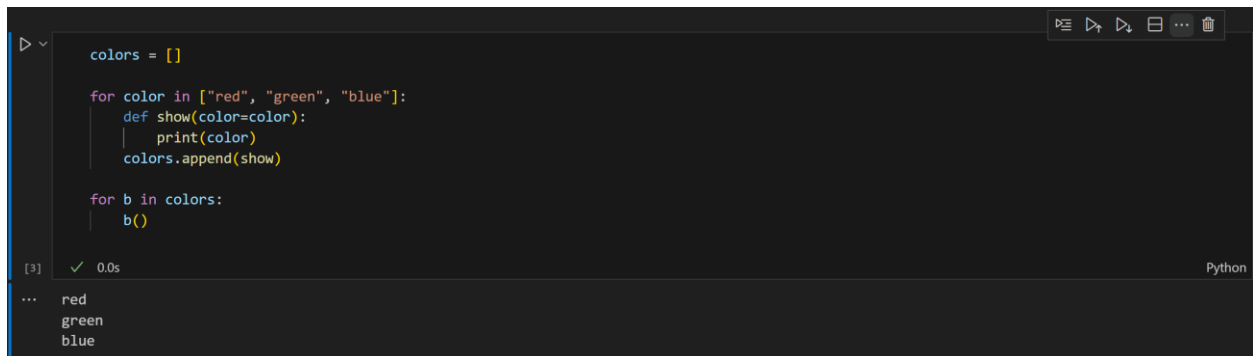
[2] ✓ 0.0s Python

... blue
blue
blue

در این کد ما در لیست `colors` سه تابع قرار دادیم که توقع داریم در هر تابع یک رنگ مجزا چاپ شود ولی خروجی که برگردانده شده نشان میدهد که همه تابع ها به ما رنگ آخری که بعنوان `color` قرار گرفته است را برمیگرداند و این مشکل از انقیاد دیر رس به وجود آمده است.

انقیاد زود رس در پایتون

در پایتون توانایی انتخاب نوع انقیاد را داریم. در قطعه کد پایین مشکل کد بالا به کمک انقیاد زود رس حل شده است:



```
colors = []

for color in ["red", "green", "blue"]:
    def show(color=color):
        print(color)
        colors.append(show)

    for b in colors:
        b()
```

[3] ✓ 0.0s Python

... red
green
blue

در این قطعه کد ما با نوشتن `color = color` به صورت دستی به برنامه گفتیم که انقیاد زود رس انجام دهد و به نوعی برای هر پرینت یک برچسب رنگ گذاشته ایم.

تبدیل نوع در پایتون

تبدیل نوع به دو شکل اصلی در زبان ها دیده میشود:

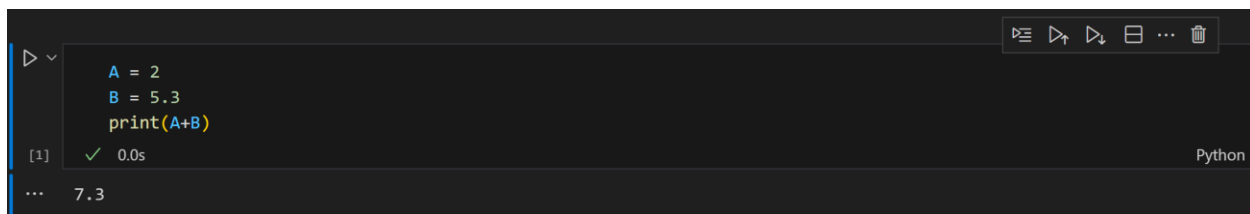
- 1 - تبدیل نوع ضمنی: این تبدیل نوع زمانی اتفاق می افتد که خود زبان به صورت خودکار و بدون دخالت برنامه نویس نوع داده ها را باهم تغییر بده تا بهم سازگار بشن.
- 2 - تبدیل نوع صریح: این تبدیل نوع زمانی است که خود برنامه نویس به صورت دستی این سازگاری رو بین نوع داده ها ایجاد میکند.

در زبان پایتون تبدیل نوع به هر دو شکل دیده میشود یعنی هم تبدیل نوع به صورت خودکار و از طریق زبان صورت میگیرد و هم اگر برنامه نویس بخواهد میتواند به صورت دستی تبدیل نوع را انجام دهد.

تبدیل نوع ضمنی در پایتون

در طراحی زبان های برنامه نویسی، تبدیل نوع ضمنی یکی از ویژگی های مهمی که مستقیماً با سهولت استفاده، انعطاف پذیری و رفتار داینامیک زبان ارتباط مستقیم دارد.

در قطعه کد زیر یک نمونه از تبدیل نوع ضمنی در پایتون رو مشاهده میکنیم:



```
A = 2
B = 5.3
print(A+B)
```

[1] ✓ 0.0s Python

... 7.3

در قطعه کد بالا مشاهده میکنیم که متغیر A از نوع int تعریف شده است و همچنین متغیر B از نوع float تعریف شده است و در قسمت print مشاهده میشود که ما دو متغیر را باهم جمع کرده ایم و با اینکه دو متغیر نوع های مختلفی دارن این عمل بدون خطا اجرا شده است. یعنی خود زبان به صورت خود کار (ضمنی) متغیر A را به float تبدیل کرده است و سپس عمل جمع را بین دو متغیر انجام داده است.

تبدیل نوع صریح در پایتون

همان طور که گفته شده بود پایتون از تبدیل نوع صریح نیز پشتیبانی میکند به این قابلیت زبان که از هر دو نوع تبدیل نوع پشتیبانی میکند (weakly typed) گفته میشود.

در قطعه کد زیر یک مثال از تبدیل نوع صریح در پایتون رو مشاهده میکنیم:

```
A, B = 2, 5.3
print(float(A)+B)
```

[2] ✓ 0.0s Python

... 7.3

در قطعه کد بالا مشاهده میکنیم که به صورت دستی و با تابع `float()` به صورت دستی متغیر `A` را از تایپ `int` به `float` تبدیل کردیم و تبدیل نوع صریح اتفاق افتاده است، یعنی به صورت دستی تبدیل نوع انجام شده است.

چالش های پشتیبانی زبان از هر دو تبدیل نوع

اینکه زبان پایتون هم از تبدیل نوع صریح و هم ضمنی پشتیبانی میکند دچار چالش ها و پیچیدگی هایی میشود که میتوان به این موارد اشاره کرد:

- 1 - جمع متغیر `bool` با `int`: در زبان پایتون عبارت `True` برابر ۱ تعریف شده است و از این رو میتواند دچار خطا های محاسباتی بشه.

```
A, B = True, 2
print(A+B)
```

[3] ✓ 0.0s Python

... 3

- 2 - کاهش شفافیت کد: پیچیدگی دیگری که این رفتار زبان ایجاد کرده است این هست که برنامه نویس دقیق نمیداند که در هر لحظه داده چه نوعی است.
- 3 - سختی تست و دیباگ: از آنجایی که در هنگام اجرای برنامه امکان دارد نوع داده ها به صورت ضمنی تغییر کنند این چالش ایجاد میشود که در هنگام تست و دیباگ برنامه نوع متغیر ها سختی در فهم برنامه ایجاد کنند

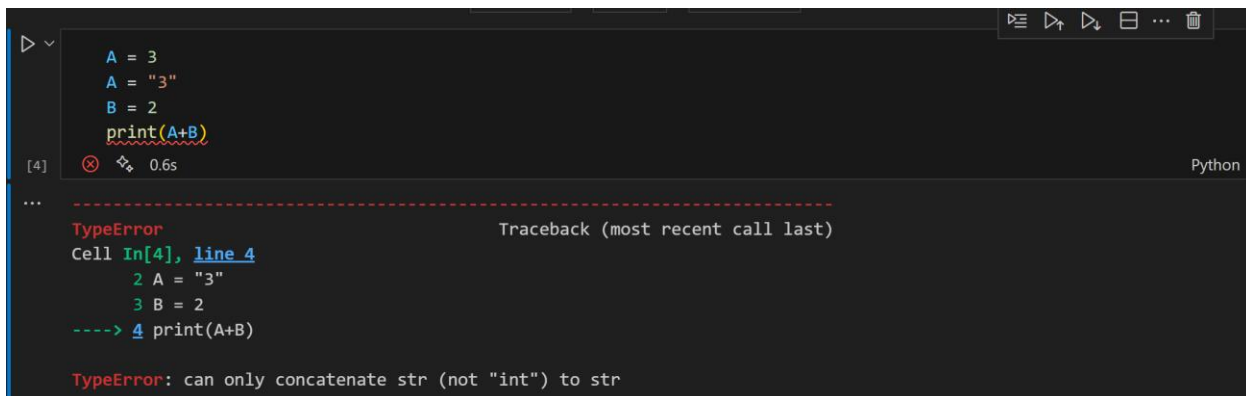
کنترل نوع در پایتون

به طور کلی کنترل نوع در زبان ها به دو دسته ایستا و پویا (static typing – dynamic typing) طبقه بندی میشوند:

- 1 - کنترل نوع ایستا: نوع متغیر ها در زمان کامپایل بررسی میشوند و هر متغیر قبل از استفاده باید نوع مشخصی داشته باشد. زبان هایی مانند C ، Java و Rust از این روش استفاده میکنند.
- 2 - کنترل نوع پویا: نوع متغیر ها در زمان اجرا بررسی میشوند. نیازی به تعریف نوع هنگام اعلان ندارند. زبان هایی که از این نوع کنترل نوع پشتیبانی میکنند میتوان اشاره کرد به: Python و JavaScript

کنترل نوع پویا در پایتون

در پایتون گفته شد که از کنترل نوع پویا استفاده میکند و این به این معنا است که در هنگام نوشتن برنامه نیازی به تعریف نوع متغیر نیست و کنترل نوع در زمان اجرای برنامه صورت میگیرد.



```
A = 3
A = "3"
B = 2
print(A+B)
```

Traceback (most recent call last)

Cell In[4], line 4

```
2 A = "3"
3 B = 2
----> 4 print(A+B)
```

TypeError: can only concatenate str (not "int") to str

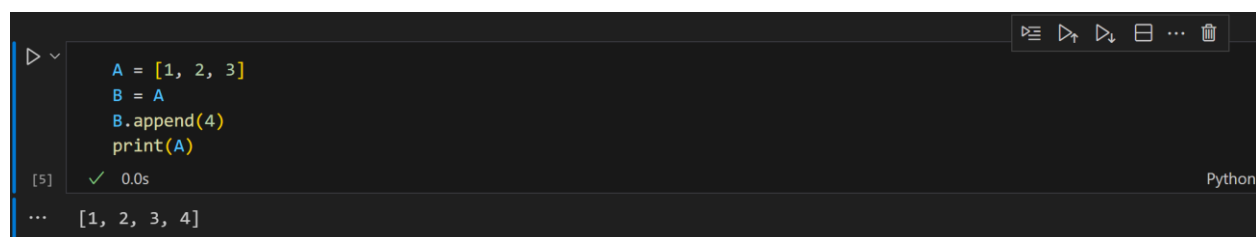
در کد بالا مشاهده میشود که در مرحله اول A از نوع int تعریف شده است ولی سپس به str تغییر پیدا کرده است و برنامه بدون مشکل این قسمت رو اجرا کرده است ولی زمانی که خواستیم این متغیر رو با یک int جمع کنیم خطای نا هماهنگی متغیر ها رو برنامه به ما نمایش داده است.

توصیف گر در پایتون

در زبان پایتون به صورت صریح اشاره گر وجود ندارد و مانند زبان هایی مثل C و C++ از علائم اشاره گر (& و *) استفاده نمیکند.

در زبان پایتون از مرجع دهی اشیاء (object references) استفاده میشود که رفتاری مشابه با اشاره گر ها رو داره یعنی زمانی که یک متغیر رو به متغیر دیگه ای نسبت میدیم در واقع هر دو به همان شیء در حافظه اشاره میکنند.

با مثال زیر درک این موضوع راحت تر میشه:



```
A = [1, 2, 3]
B = A
B.append(4)
print(A)
```

[5] ✓ 0.0s Python

... [1, 2, 3, 4]

در کد بالا یک لیست به نام A تعریف کردیم و سپس متغیر B رو به A نسبت دادیم. در این حالت B دقیقاً به همان جایی از حافظه اشاره میکند که A اشاره میکند و در واقع کپی از روی A صورت نگرفته شده است. در خط سوم ما به لیست B یک عنصر اضافه میکنیم و مشاهده میکنیم که با نمایش لیست A مقدار جدید به این لیست هم اضافه شده است. این رفتار برای ما کاملاً شفاف میکند که هر دو لیست A و B به یک جا از حافظه اشاره میکنند.

نوع شمارش در پایتون

نوع شمارش یا (Enumeration) در زبان ها به شیوه های گوناگون تعریف شده است مثلاً در زبان C با کلیدواژه (enum) میتوانیم که نوع شمارشی از داده رو تولید بکنیم یا در زبان java این عمل به صورت کلاس تعریف شده است و با خواندن کلاس (enum) نوع شمارشی میتوان تولید کرد.

در زبان پایتون برای ساخت نوع شمارشی از کتابخانه (enum) و متود (Enum) استفاده میکنیم:

```
from enum import Enum

class colors(Enum):
    red = 1
    green = 2
    blue = 3
```

در قطعه کد بالا در مرحله اول متود (Enum) به برنامه اضافه شد و سپس یک کلاس فرزند ساخته شده است که والد این کلاس، کلاس (Enum) قرار داده شده است و به این شکل میتوان در زبان پایتون نوع شمارشی خود را ایجاد کنید.

علاوه بر کلاس (Enum) میتوان از تابع (enumerate) که به صورت پیشفرض در زبان پایتون تعریف شده است استفاده کرد.

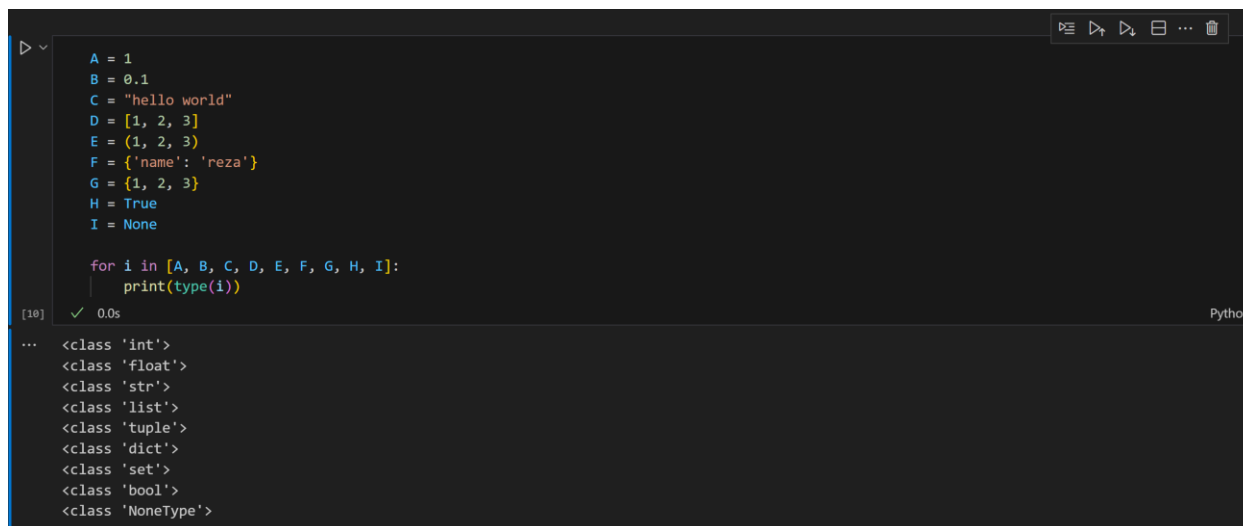
```
for A, B in enumerate(["red", "green", "blue"]):
    print(A, B)
```

[8] ✓ 0.0s

... 0 red
1 green
2 blue

انواع رشته های ورودی در پایتون

زبان پایتون بعنوان یک زبان سطح بالا و داینامیک انواع داده های زیادی رو پشتیبانی میکند این نوع داده ها عبارت است از:



```
A = 1
B = 0.1
C = "hello world"
D = [1, 2, 3]
E = (1, 2, 3)
F = {'name': 'reza'}
G = {1, 2, 3}
H = True
I = None

for i in [A, B, C, D, E, F, G, H, I]:
    print(type(i))
```

[10] ✓ 0.0s Python

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
<class 'bool'>
<class 'NoneType'>
```

در بین نوع داده های گفته شده، لیست یکی از پر کاربرد ترین نوع داده های پایتون است که میتواند بعنوان هر عضو خود هر نوع داده ای رو داخل خودش نگهداری بکند.

در واقع رفتار لیست مانند یک آرایه از نشانگر ها است که هر نشانگر به یک فضای خالی در حافظه اشاره میکند و زمانی که داده جدیدی به لیست اضافه کنیم، یک نشانگر از لیست به داده جدید اشاره میکند.

کپسول سازی در پایتون

یکی از چهار اصل برنامه نویسی پویا کپسول سازی یا (Encapsulation) هست و به این هست که در کلاسی که نوشته شده است چه متود هایی دسترسی عمومی دارند و چه متود هایی دسترسی خصوصی دارند.

دسترسی خصوصی (private): این متود ها فقط از داخل کلاس حق دسترسی دارن و خارج از کلاس قابل فراخوانی نیستند.

دسترسی عمومی (public): این دسته از متود ها علاوه بر دسترسی از داخل خود کلاس، خارج کلاس نیز فراخوانی میشوند.

در زبان پایتون برای نوشتن اسم متود خصوصی پیشوند (__) استفاده میشود و زبان از این مورد متوجه میشود که متود نوشته شده از نوع خصوصی (private) است. اما برای نوشتن متود عمومی (public) نیاز به پیشوندی ندارد و اسم متود به صورت عادی نوشته میشود.

در قطعه کد زیر تعریف این دو دسته از سطح دسترسی رو در پایتون میبینیم:

```
class test():
    def __private(self):
        print("private")

    def public(self):
        print("public")

test1 = test()
test1.public()
test1.__private()
```

[12] 0.0s Python

... public

... -----

AttributeError Traceback (most recent call last)

Cell In[12], line 10

8 test1 = test()

9 test1.public()

--> 10 test1.__private()

AttributeError: 'test' object has no attribute '__private'

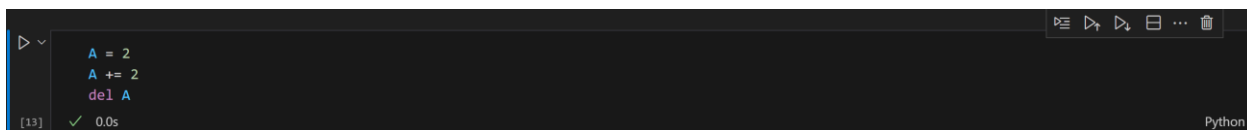
همانطور که در قطعه کد میبینیم تابع عمومی بدون هیچ خطایی اجرا شده است ولی زمانی که خواستیم متود خصوصی رو خارج از کلاس خودش فراخوانی کنیم به ارور برخورد کردیم.

مدیریت زباله در پایتون

یکی از نکاتی که برنامه نویس در هنگام نوشتن برنامه خود باید دقت کند مدیریت متغیر هایی است که دیگر کاری با آنها ندارند. اگر برنامه نویس برای مدیریت این متغیر ها کاری انجام ندهد با جلو رفتن و بزرگ شدن برنامه، باعث کندی در برنامه و حجیم شدن برنامه میشود و در مواقعی باعث آسیب پذیری و هک برنامه میشوند.

تقریباً در همه زبان ها دستوری پیش فرض برای حذف متغیر ها وجود دارد. در زبان CPP این دستور delete نام دارد و در زبان مورد بحث ما، Python این دستور del هست.

در زبان پایتون با دستور del میتوانیم که متغیر مورد نظر رو به صورت کامل از برنامه حذف کنیم و فضایی که این متغیر اشغال کرده است رو آزاد کنیم.

A screenshot of a Python interpreter window. The code being executed is:

```
A = 2  
A += 2  
del A
```

 The output shows a green checkmark and the text "[13] ✓ 0.0s". The word "Python" is visible in the bottom right corner of the window.

در این مثال ما متغیر A رو با دستور del حذف کردیم و حافظه ای که متغیر A اشغال کرده بود از این پس آزاد است و میتواند مجدد متغیر جدید به این آدرس از حافظه اشاره کند.

نگهداری رکورد ها در پایتون

نگهداری رکورد هایی که حجم بالایی دارند در زبان پایتون کار آسانی است چون از دو نوع داده قدرتمند List و Dictionary پشتیبانی میکند.

برای نگهداری داده هایی که ترتیب قرار گیری شان مهم هست مانند سری زمانی میتوان از لیست استفاده کرد این نوع داده علاه و بر قابل شمارش بودن متود های کاربردی مانند append، drop، insert و ... در خود دارد.

از سمت دیگر برای نگهداری که حالت key، value دارند نوع داده دیکشنری وجود دارد بطور مثال برای نگهداری اطلاعات هر استان کشور میتوان از این نوع داده استفاده کرد. دیکشنری هم مانند دیگر نوع داده های پایتون متود های کاربردی رو به برنامه نویسان ارائه کرده است.

زبان پایتون برای ذخیره و بازخوانی رکورد ها از انواع فایل ها کتابخانه هایی دارد که میتواند با این کتابخانه ها رکورد های داخل برنامه رو در انواع فایل ها ذخیره کرد.

```
import json
import xlr
import csv
import h5py
import pickle
```

Python

ساختار لیست در پایتون

در زبان ها انواع لیست ها وجود دارد بطور مثال کاربردی ترین لیست در زبان CPP، Array List هست که یک دنباله از یک نوع داده رو داخل خودش ذخیره میکند. این شکل از لیست طول ثابتی داره و در زمان تعریف باید تعداد فضایی که قرار است این لیست داشته باشد رو مشخص کرد.

نوع دیگر لیست Linked Lists هست که توی هر خونه از لیست های پیوندی دو مقدار وجود دارد. یک، مقداری که باید اون خونه از حافظه در خود نگهداری کند. دو، یک اشاره گر که آدرس بعدی خونه حافظه رو نشان میده. نکته ای که در این لیست وجود دارد این هست که میتوان در هر خانه یک نوع داده ذخیره کرد ولی همچنان مانند آرایه ها طول این لیست ها ثابت است.

نوع سوم لیست ها که پایتون نیز از این نوع استفاده میکند Dynamic Lists هست. ویژگی این لیست نسبت به لیست های پیوندی این هست که علاوه بر ذخیره نوع های مختلف در یک لیست، داینامیک لیست توانایی تغییر طول خود را دارد و پایتون این عمل را با متود هایی که برای داینامیک لیست تعریف کرده است میتواند اعمال زیادی روی این لیست ها انجام دهد.

نوع چهارمی از لیست ها وجود دارد که در واقع از ترکیب دو لیست به وجود می آید. یعنی یک لیست داریم که هر خانه از این لیست به یک لیست دیگر اشاره میکند اسم این گونه از لیست ها Nested Lists هست. این مدل از لیست رو هم با لیست های پیوندی میتوان درست کرد و هم با داینامیک لیست.

در ادامه انواع متود هایی که برای داینامیک لیست در زبان پایتون وجود دارد رو مشاهده میکنیم:

```
print(dir(list))
```

[9] ✓ 0.0s Python

... '_sizeof_', '_str_', '_subclasshook_', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort'